Interview Task #5: Live Auction Bidding Platform — "BidBlitz"

General Information

Field	Details
Level	Senior Frontend Developer
Core Topics	Event-Driven Architecture, Race Conditions, Countdown Synchronization, Async Bid Validation, Unit Testing
Allowed Tools	Any AI assistant, any documentation, any npm package. You must document ALL AI prompts in `PROMPTS.md`
Estimated Time	4–8 hours (half day to full day)
Stack	Vanilla JavaScript (or any framework) + provided Node.js mock server

User Story

The Scenario:

Your friend Noa runs a small vintage shop and wants to ditch the shouting-based auction nights she hosts every Thursday in her garage. Last week, two regulars almost got into a fight over a 1987 Nintendo — both claimed they bid 350 first. Noa wants a simple web app where people can bid on items in real-time, see who's winning, and — most importantly — where the system decides who won, not Noa's ears.

> *"I just need something that shows people what's up for auction, lets them bid, and when the timer runs out — boom — we know who won. No more garage drama."* — Noa

Task Description

Build a **Live Auction Bidding Dashboard** that connects to the provided mock server. Users can browse active auctions, place bids in real-time, and watch countdowns tick to zero — at which point the winner is revealed. The server broadcasts bid updates via **Server-Sent Events (SSE)** and uses deliberate quirks to simulate real-world problems you'll need to handle.

Mock Server Setup

Copy the server code below into `server.js` and run with `node server.js`. The server runs on port **3005**.

```
const http = require('http');
const url = require('url');

// ============ AUCTION DATA ============
const auctions = [
  { id: 'a1', title: '1987 Nintendo NES Console', image: '🎮', startPrice: 100, currentBid: 100, currentBidder:
null, endsAt: Date.now() + 180000, status: 'active', bidHistory: [] },
  { id: 'a2', title: 'Vintage Polaroid Camera', image: '📸', startPrice: 50, currentBid: 50, currentBidder: nul
l, endsAt: Date.now() + 240000, status: 'active', bidHistory: [] },
  { id: 'a3', title: 'First Edition Harry Potter', image: '📚', startPrice: 200, currentBid: 200, currentBidder:
null, endsAt: Date.now() + 120000, status: 'active', bidHistory: [] },
  { id: 'a4', title: 'Signed Beatles Vinyl', image: '🎵', startPrice: 500, currentBid: 500, currentBidder: null,
endsAt: Date.now() + 300000, status: 'active', bidHistory: [] },
  { id: 'a5', title: 'Retro Arcade Machine', image: '👾', startPrice: 150, currentBid: 150, currentBidder: null,
endsAt: Date.now() + 60000, status: 'active', bidHistory: [] },
  { id: 'a6', title: 'Antique Pocket Watch', image: '🕰️', startPrice: 75, currentBid: 75, currentBidder: null, e
ndsAt: Date.now() + 360000, status: 'active', bidHistory: [] },
];

let sseClients = [];
let bidSequence = 0;

// ============ SERVER QUIRKS (Documented for evaluators) ============
// QUIRK 1: POST /api/bid responds after 800-2500ms delay (simulates validation)
// QUIRK 2: SSE stream sometimes sends duplicate events (same bid_id twice)
// QUIRK 3: POST /api/bid returns 409 if auction ended between request and processing
// QUIRK 4: SSE events may arrive OUT OF ORDER (bid #5 before bid #4)
// QUIRK 5: Server sends "heartbeat" events every 15s — client must ignore these
// QUIRK 6: Every ~45s the SSE connection drops and must be re-established
// QUIRK 7: GET /api/auctions has a 30% chance of 500ms extra latency

function broadcast(event, data) {
  const id = ++bidSequence;
  const msg = `id: ${id}\nevent: ${event}\ndata: ${JSON.stringify(data)}\n\n`;
  sseClients.forEach(res => {
    try { res.write(msg); } catch (e) { /* client disconnected */ }
  });
  // QUIRK 2: 20% chance of duplicate event (same data, same id)
  if (Math.random() < 0.2) {
    setTimeout(() => {
      sseClients.forEach(res => {
        try { res.write(msg); } catch (e) {}
      });
    }, 100 + Math.random() * 400);
  }
  // QUIRK 4: 15% chance next event arrives before this one for new clients
  // (simulated by sometimes delaying the original)
}

// Check and close expired auctions
setInterval(() => {
  auctions.forEach(a => {
    if (a.status === 'active' && Date.now() >= a.endsAt) {
      a.status = 'ended';
      broadcast('auction_ended', {
        auctionId: a.id,
```

```
        title: a.title,
        winner: a.currentBidder,
        finalPrice: a.currentBid,
        timestamp: Date.now()
      });
    }
  });
}, 1000);

// QUIRK 5: Send heartbeat every 15 seconds
setInterval(() => {
  const msg = `event: heartbeat\ndata: ${JSON.stringify({ timestamp: Date.now() })}\n\n`;
  sseClients.forEach(res => {
    try { res.write(msg); } catch (e) {}
  });
}, 15000);

// QUIRK 6: Drop SSE connections every ~45 seconds
setInterval(() => {
  sseClients.forEach(res => {
    try { res.end(); } catch (e) {}
  });
  sseClients = [];
}, 45000);

// Simulated competing bidders (other "garage attendees")
setInterval(() => {
  const activeAuctions = auctions.filter(a => a.status === 'active');
  if (activeAuctions.length === 0) return;
  const auction = activeAuctions[Math.floor(Math.random() * activeAuctions.length)];
  const botNames = ['GarageGuy_Ron', 'VintageVicki', 'BidMaster3000', 'NostalgiaNate'];
  const botName = botNames[Math.floor(Math.random() * botNames.length)];
  const increment = [5, 10, 15, 20, 25][Math.floor(Math.random() * 5)];
  const newBid = auction.currentBid + increment;
  auction.currentBid = newBid;
  auction.currentBidder = botName;
  auction.bidHistory.push({ bidder: botName, amount: newBid, timestamp: Date.now() });
  broadcast('new_bid', {
    auctionId: auction.id,
    bidder: botName,
    amount: newBid,
    previousBid: newBid - increment,
    timestamp: Date.now(),
    bid_id: `bid_${Date.now()}_${Math.random().toString(36).substr(2, 5)}`
  });
}, 8000 + Math.random() * 7000);

const server = http.createServer((req, res) => {
  const parsedUrl = url.parse(req.url, true);
  const path = parsedUrl.pathname;

  // CORS
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, OPTIONS');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type');
  if (req.method === 'OPTIONS') { res.writeHead(204); res.end(); return; }
```

```
  // GET /api/auctions — list all auctions
  if (path === '/api/auctions' && req.method === 'GET') {
    // QUIRK 7: 30% chance of extra 500ms latency
    const delay = Math.random() < 0.3 ? 500 : 0;
    setTimeout(() => {
      res.writeHead(200, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify(auctions.map(a => ({
        id: a.id, title: a.title, image: a.image,
        startPrice: a.startPrice, currentBid: a.currentBid,
        currentBidder: a.currentBidder, endsAt: a.endsAt,
        status: a.status, bidCount: a.bidHistory.length
      }))));
    }, delay);
    return;
  }

  // GET /api/auctions/:id — single auction with bid history
  if (path.match(/^\/api\/auctions\/\w+$/) && req.method === 'GET') {
    const id = path.split('/').pop();
    const auction = auctions.find(a => a.id === id);
    if (!auction) { res.writeHead(404); res.end(JSON.stringify({ error: 'Auction not found' })); return; }
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify(auction));
    return;
  }

  // POST /api/bid — place a bid
  if (path === '/api/bid' && req.method === 'POST') {
    let body = '';
    req.on('data', chunk => body += chunk);
    req.on('end', () => {
      // QUIRK 1: Artificial delay 800-2500ms
      const delay = 800 + Math.random() * 1700;
      setTimeout(() => {
        try {
          const { auctionId, bidder, amount } = JSON.parse(body);
          if (!auctionId || !bidder || !amount) {
            res.writeHead(400);
            res.end(JSON.stringify({ error: 'Missing required fields: auctionId, bidder, amount' }));
            return;
          }
          const auction = auctions.find(a => a.id === auctionId);
          if (!auction) { res.writeHead(404); res.end(JSON.stringify({ error: 'Auction not found' })); return; }
          // QUIRK 3: 409 if auction ended during the delay
          if (auction.status === 'ended') {
            res.writeHead(409);
            res.end(JSON.stringify({ error: 'Auction has ended', winner: auction.currentBidder, finalPrice: aucti
on.currentBid }));
            return;
          }
          if (amount <= auction.currentBid) {
            res.writeHead(400);
            res.end(JSON.stringify({ error: 'Bid must be higher than current bid', currentBid: auction.currentBid
}));
            return;
          }
          const bid_id = `bid_${Date.now()}_${Math.random().toString(36).substr(2, 5)}`;
```

```
          auction.currentBid = amount;
          auction.currentBidder = bidder;
          auction.bidHistory.push({ bidder, amount, timestamp: Date.now() });
          broadcast('new_bid', {
            auctionId: auction.id,
            bidder,
            amount,
            previousBid: amount,
            timestamp: Date.now(),
            bid_id
          });
          res.writeHead(200, { 'Content-Type': 'application/json' });
          res.end(JSON.stringify({ success: true, bid_id, currentBid: amount, message: `Bid of ${amount} placed s
uccessfully` }));
        } catch (e) {
          res.writeHead(400);
          res.end(JSON.stringify({ error: 'Invalid JSON' }));
        }
      }, delay);
    });
    return;
  }

  // GET /api/stream — SSE stream for real-time updates
  if (path === '/api/stream' && req.method === 'GET') {
    res.writeHead(200, {
      'Content-Type': 'text/event-stream',
      'Cache-Control': 'no-cache',
      'Connection': 'keep-alive',
    });
    res.write(`event: connected\ndata: ${JSON.stringify({ message: 'Connected to BidBlitz stream', timestamp: Dat
e.now() })}\n\n`);
    sseClients.push(res);
    req.on('close', () => { sseClients = sseClients.filter(c => c !== res); });
    return;
  }

  res.writeHead(404);
  res.end(JSON.stringify({ error: 'Not found' }));
});

server.listen(3005, () => console.log('BidBlitz Mock Server running on http://localhost:3005'));
```

Basic Requirements

1. Auction Catalog View

Requirement	Details
Fetch and display all auctions	`GET /api/auctions` — show title, image emoji, current bid, bidder, and time remaining
Countdown timers	Each auction card must show a **live countdown timer** that ticks every second (mm:ss format)
Visual status indicators	Active auctions show green, auctions under 30s show red/pulsing, ended auctions show gray with winner
Click to view details	Clicking an auction opens a detail view with full bid history from `GET /api/auctions/:id`

2. Real-Time Bid Updates (SSE)

Requirement	Details
Connect to SSE stream	`GET /api/stream` — listen for `new_bid` and `auction_ended` events
Live bid updates	When a `new_bid` event arrives, update the relevant auction card immediately (price + bidder name)
Auction end handling	When `auction_ended` fires, show winner overlay/banner on that auction card, stop the countdown
Handle heartbeat events	Server sends `heartbeat` events every 15s — your app must **ignore** these gracefully

3. Place a Bid

Requirement	Details
Bid form	User enters their name and bid amount; send `POST /api/bid` with `{ auctionId, bidder, amount }`
Minimum bid validation	Client-side: bid must be higher than displayed current bid. Show error if not.
Loading state	Show loading/spinner while bid is being processed (server has 800-2500ms delay)
Error handling	Handle 400 (bid too low — someone outbid you during delay) and 409 (auction ended) with clear user messages

4. Race Condition Handling

Requirement	Details
Outbid during delay	If a user submits a bid of 200 but a bot bids 210 during the server delay, the user gets a 400. Handle this gracefully — update the displayed bid and suggest rebidding.
Auction ends during bid	If the countdown hits zero while the user's bid is in-flight, the server returns 409. Show "Auction ended" — don't show a success message.
Duplicate SSE events	Server sometimes sends the same `new_bid` event twice (same `bid_id`). Your app must deduplicate — don't show the same bid twice in history.

5. SSE Connection Management

Requirement	Details
Auto-reconnect	Server drops SSE connection every ~45s. Your app must detect disconnection and reconnect automatically.
Connection status indicator	Show a visual indicator: Connected / Disconnected / Reconnecting
Data reconciliation	After reconnecting, fetch fresh auction data to catch any bids missed during the disconnect gap.

6. Unit Tests

Requirement	Details
Minimum 5 tests	Use any testing library (Jest, Vitest, or even manual test file with assertions)
Must test	At least: bid validation logic, duplicate event detection, countdown calculation, auction status transitions
Testable code structure	Logic must be separated from DOM manipulation so it can be unit tested

---

Bonus Features

Feature	Description	Tag
Bid Snipe Protection	If a bid comes in with <10s remaining, extend the countdown by 15s (show visual indicator that time was extended)	UX
Bid History Chart	Show a simple price-over-time chart for each auction's bid history	Visual
Sound/Visual Notification	Play a sound or flash when the user is outbid on an auction they've bid on	UX
My Bids Tracker	A sidebar showing all auctions the user has bid on, with "winning" / "outbid" / "won" / "lost" status	Feature

Technical Requirements

Requirement	Details
No full-page reloads	All interactions must be SPA-style (fetch API, DOM updates)
PROMPTS.md	Document every AI prompt you used, what you expected, what you got, and what you changed. **This is graded.**
Clean code structure	Separate concerns: API layer, event handling, DOM rendering, state management
Error boundaries	No unhandled promise rejections, no silent failures. All errors must be visible to the user or logged.
Countdown accuracy	Timers must be based on server `endsAt` timestamps, not client-side guessing. Handle clock drift.

Recommended Execution Order

Step	Task	Est. Time
1	Set up project, run mock server, explore API endpoints manually	20 min
2	Build auction catalog view with countdown timers	60 min
3	Connect SSE stream, handle `new_bid` and `auction_ended` events	45 min
4	Implement bid form with loading states and error handling	45 min
5	Handle race conditions: outbid during delay, 409, duplicate events	45 min
6	SSE reconnection + data reconciliation	30 min
7	Write unit tests	30 min
8	Polish UI, write README, finalize PROMPTS.md	30 min
9	(Bonus) Bid snipe protection, charts, notifications	remaining time

Submission Instructions

Submit a ZIP or Git repository containing:

1. All source code files
2. `server.js` (the mock server — unchanged)
3. `README.md` — How to run, architecture decisions, known limitations
4. `PROMPTS.md` — All AI prompts used, with reflections on what worked and what didn't
5. Test files
6. `package.json` if using npm packages

Tips

Read the server code! The quirks are documented in comments. Understanding them before coding will save you hours.

Start with the happy path. Get auctions displaying with timers first. Add SSE second. Add bidding third. Handle edge cases last.

The server has bots. Other "bidders" place bids automatically every 8-15 seconds. Use this to test your real-time updates without needing to open multiple tabs.

Timer accuracy matters. Use `endsAt` from the server, not `setInterval` counting down from a snapshot. Think about what happens if the user's tab is in the background for 30 seconds.

Test your reconnection. The server drops SSE connections every ~45 seconds — you'll see this naturally while developing. Make sure your app handles it smoothly.

Document your thinking. A good PROMPTS.md is worth more than a perfect UI.

What We Evaluate

Your submission will be assessed across six dimensions. Make sure your work demonstrates all of them.

1. **Spec Driven Development (SDD)** — How you broke down requirements before coding, your use of AI tools documented in `PROMPTS.md`, whether you iterated and critically evaluated AI output rather than accepting it blindly, and whether your final solution traces back to the original requirements.
1. **Architecture** — How you structured your code: separation between API layer, state management, event handling, and DOM rendering. Reusable modules, single-responsibility functions, and a coherent approach to managing shared state across auction cards.
1. **Security** — Input sanitization on bidder name and bid amount before sending to the server, prevention of XSS through safe DOM insertion (avoid `innerHTML` with unescaped user data), proper handling of untrusted SSE data, and no exposure of sensitive state in client-side globals.
1. **Performance** — Efficient DOM updates (do not re-render the entire auction list on every SSE event), proper cleanup of `setInterval` timers when auctions end or the component unmounts, avoidance of memory leaks from accumulated SSE event listeners, and responsive countdown rendering without jank.
1. **Observability** — Meaningful console logging that helps diagnose connection drops, bid failures, and deduplication events without flooding the console with noise. Error states must be visible to the user (not swallowed silently). Connection status indicator gives real-time feedback on SSE health.
1. **Tests** — At least 5 unit tests covering core logic: bid amount validation, duplicate `bid_id` detection, countdown calculation from `endsAt`, and auction status transitions. Test logic must be separated from DOM code so tests can run without a browser.
